Department of Education

STEM Education

National Schools of Excellence

Technology Teacher's Guide

Grade 12

Term 1

Science
Technology
Engineering
Mathematics

Issued free to schools by the Department of Education

Published in 2021 by the Department of Education, Papua New Guinea

ISBN …………………..

Acknowledgements

Ultima Thulë Ltd was engaged by the Department of Education to develop the STEM Curriculum. It was written by writers from universities, secondary schools, new university graduates with expertise in the subject matter and industry subject experts. Quality assurance groups and the Science Subject Advisory Committee have also contributed to the development of this syllabus.

This document was developed with support from the General Education Services Division of the Department of Education.

Table of Contents

SECRETARY'S MESSAGE

This Technology syllabus is to be used exclusively by teachers of Technology to teach Grade 12 Students enrolling for the STEM Education through the Schools of Excellence throughout Papua New Guinea. The STEM Technology Bridging Syllabus ensured that Grade 11 students enrolling for STEM Education were grounded in the basic concepts espoused in Technology before undertaking the advanced concepts introduced in this syllabus.

The syllabus conforms to the visions of the National Education Plan and the School of Excellence Policy. Furthermore, it is undergirded by a policy and legislative matrix that were issued by the National Government over the last decades, such as the Vision 2050 and Development Strategic Plan 2030. The Preamble of the National Constitution, especially the First Directive of the *National Goals and Directive Principles* makes a clarion call for the Integral Human Development, which the Department took heed of to ensure, through this new syllabus, that *all forms of beneficial creativity, including sciences and cultures, [are] to be actively encouraged*. A principle object of the *National Education Act 1983 (amended 1995)* reiterates this virtue of Integral Human Development (Section 4).

In accordance with the widespread demand for a home-grown curriculum to replace the existing one with the view to position the country as a bastion of innovation in the new century the Secretary of Education has embarked on introducing the STEM Curriculum to the Schools of Excellence. It is hoped that students coming through the STEM Education will not only gain productive employment in the knowledge-based economy but become creators, innovators and inventors of knowledge and ultimately commercialize their creativity through development of their intellectual property rights.

In addition, there are four core elements espoused in this syllabus, insofar as student development is concerned: Students are expected to gain knowledge and skills and apply them to solve every day problems whilst appreciating Technology as a fundamental field of science.

This STEM Technology Syllabus for Grade 12 constitutes eight (8) Modules and is intended for students enrolling for STEM Education. In this booklet only three Thema are presented. By studying these Thema students are expected to gain the foundational knowledge needed in Computer Science.

The success of this new Curriculum is highly dependent on the Teachers. They play a pivotal role by being innovative and keeping abreast of new information on scientific research and innovations. The challenge for teachers of Technology is to engage students to learn realistic context for increased and better understanding and appreciate the relevance of Technology to human lives, including good health and environment.

In order to ensure Teachers discharge their responsibilities adequately this syllabus is accompanied by a Teacher's Guide and Teacher's Resource Book which provides sufficient teaching and assessment materials to help Teachers.

I commend and approve this syllabus as the official curriculum for Technology to be used for the Grade 12 STEM Education Program in the Schools of Excellence throughout Papua New Guinea.

DR UKE KOMBRA, PHD
Secretary for Education

INTRODUCTION

The Technology Syllabus introduced to the Schools of Excellence as a critical component of the STEM Education is mostly concerns with the Information, Communication and Technology (ICT) and Computer Science. Although, ICT has been part of the existing Upper Secondary Syllabi, it has not been taught in a structured approach as it is intended in this Syllabus. The technology space, in general, is emerging at a much faster pace than the rest of the STEM courses and this necessitates the need to introduce the course in a structured way going forward.

Teaching Technology requires laboratory sessions as an essential part of the syllabus. Practical work can be used to enable students to investigate the properties and reactions of substances and to provide opportunities for students to learn and test the principles and concepts of Computer Science and ICT.

The Grade 11 Technology Syllabus entails ten (10) Thematic Areas (Thema) and these are listed below. Each Thema has a single Module and three topics. Over time different Modules will be introduced.

MODULE	TOPICS
12.1 JavaScript	1. Basic JavaScript 2. JavaScript Objects 3. Advance JavaScript
12.2 Hypertext Preprocessor (PHP)	1. Basic PHP 2. Advance PHP
12.3 Software Engineering	1. Software Development Life Cycle (SDLC) 2. PP vs OOP 3. Software Applications and Problem Solving
12.4 Python	1. Basic Python 2. Advance Python
12.5 Java	1. Basic Java 2. Java Object-Oriented 3. Advance Java
12.6 C/C++ Programming	1. Basic C++ 2. Object-Oriented C++ 3. Advance C++ 4. Basic C Programming
12.7 Artificial Intelligence	1. Introduction to Artificial Intelligence (AI) 2. Mathematics of Artificial Intelligence (AI) 3. Science of Artificial Intelligence (AI) 4. Machine Learning 5. Data Mining
12.8 Robotics	1. Introduction to Robotics 2. Basic Robotics Programming 3. LISP 4. MATLAB 5. Programming a Robot

Since Computer Science is a highly specialized subject students undertaking Technology are expected to possess high levels of cognitive and numeracy competency, and a firm grasp of the language skills.

The modules in this Syllabus are sequenced to guide teachers of Technology through the process of imparting basic Computer Science knowledge and skills to students

RATIONALE

There is a confluence of events that necessitated the need to introduce STEM Curriculum to the Schools of Excellence:

1. The School of Excellence Policy that was launched by Prime Minister Honourable James Marape, MP, in June 2020, calls for the establishment of the Schools of Excellence and the introduction of STEM Curriculum;
2. The Government has decided to replace the Observation Based Education to Standards Based Education in 2019 and tasked the Department of Education to introduce the STEM Curriculum as part of its reform agenda;
3. There is public outcry, both from public and private sectors, over the quality of graduates coming through the National Education System; and
4. The changing landscape in the technology domain globally and the pressing need to produce the next generation of workforce that is agile and can adopt to the changing circumstances as well as become enterprising individuals through their creation of intellectual properties.

The combination of these factors has led the Department of Education to adopt the STEM Curriculum and to introduce this to the Schools of Excellence as a necessary disruptive intervention in an effort to develop the next generation of national intellectual capital and to drive the growth and prosperity of the country through creation of intellectual properties.

In developing the STEM Curriculum, it was realized that the Grades 9 and 10 Science Syllabi does not provide a firm foundation to launch students into STEM Curriculum at the Schools of Excellence. Consequently, a STEM Bridging Program was necessary. The STEM Bridging Syllabus is a bridge across the divide that Grade 11 Students enrolling for STEM Education must cross.

The Technology Syllabus was introduced as a Bridging Course to the Schools of Excellence. The syllabus introduced the students to the Technology Domain and expects them to develop an appreciation of the discipline in a broader context. Students learned the basic concepts espoused in only two phyla of Technology – Information Technology and Computer Science. In this syllabus students are introduced to advanced concepts in Computer Science as they are applied to the ten Thema.

The introduction of the STEM Curriculum, especially the Bridging Program and the Grades 11 and 12 Syllabi, will cause consequential changes to the Upper Classes of STEM Education and the Lower Secondary Syllabi. These syllabi will be introduced in the course of time.

STEM Domain

At this juncture of our national development there is no other task before us more important than deploying a robust STEM Curriculum that develops the next generation of highly educated students to enter the work force. By building on the best of current practices and standards the STEM Curriculum aims to take us, beyond the constraints of the existing structures of schooling, towards a shared vision of excellence nationally and marketing the nation globally in terms of Science, Technology, Engineering and Mathematics. In this STEM Curriculum, there are ten (10) Thema (Thematic Areas) introduced.

Science

It is the study of the natural world, including the laws of the nature associated with physics, chemistry and biology and the treatment or application of facts, principles, concepts or conventions associated with these disciplines. Science is both a body of knowledge that has been accumulated over time and a process-scientific inquiry-that generates new knowledge. Knowing from science informs the engineering design process. In this STEM Curriculum the Science Domain has three Phyla – Biology, Chemistry and Physics.

Technology

It comprises the entire system of people and organizations, knowledge, processes, and devices that go into creating and operating technological artifacts, as well as the artifacts themselves. Throughout history humans have created technology to satisfy their wants and needs. Much of modern technology is the product of science and engineering and technological tools used in both fields. There are two Phyla – Information Technology and Computer Sciences – introduced in this STEM Curriculum for the Technology Domain.

Engineering

It concerns both the knowledge about the design and creation of products and a process for solving problems. The design process must result in a final product that must operate or function under various constraints. One constraint in engineering design is the laws of nature. Engineers must learn to harness the natural forces at work around them and through their design process develop products that must operate in harmony with nature. Other constraints include things such as time, money, available materials, ergonomics, environmental regulations and manufacturability. Engineering utilizes concepts in science and mathematics as well as technological tools. There are three (3) Phyla introduced in the Engineering Domain – Civil, Mechanical and Electrical.

Mathematics

It is the study of patterns and relationships among quantities, numbers, and shapes. Specific branches of mathematics include arithmetic, geometry, algebra, trigonometry and calculus. Mathematics is used in science and in engineering. In this STEM Curriculum, Mathematics is considered as a Service Domain; specific topics tailored towards specific Thema are also considered relevant to undergird the relevance and applicability of Mathematics Domain to other fields of knowledge.

STEM TECHNOLOGY CONTENT OVERVIEW – GRADE 12

Table 1.1: Sequencing & Benchmarking STEM Technology Content Standards

<table>
<tr><td colspan="4">Setting STEM Technology Content Standards for Progressive Learning and Assessment National STEM Assessment</td></tr>
<tr><td colspan="4">DOMAIN: Technology</td></tr>
<tr><td>MODULE 12.1:
JavaScript</td><td colspan="3">Content Standard:
12.1: Utilizing JavaScript basic-to-advanced concepts on Web Development applications</td></tr>
<tr><td>Topic</td><td>Performance Indicators</td><td>Module Benchmark</td><td>Assessment Tasks</td></tr>
<tr><td rowspan="2">1. JavaScript (Basic)</td><td>12.1.1a: Enabling and disabling JavaScript support in browsers

12.1.1b: Placing JavaScript in HTML files in different ways

12.1.1c:Utilizing and/or manipulate the different JavaScript data types to support a software program

12.1.1d: Declare and utilize variables with JavaScript

12.1.1e: Write decision-making code using if…else…elseif conditional statements in JavaScript

12.1.1f: Use switch…case statement to test or evaluate an expression with different values in JavaScript.

12.1.1g: Manipulate or perform the operations on variables and values using the operators in JavaScript</td><td rowspan="2">Students' ability to master the knowledge and skills of:
Creating responsive and interactive elements of a Webpage/Website using programming skills of HTML, CSS and JavaScript.</td><td rowspan="2">Assignment + Test + Quiz</td></tr>
<tr><td>`
12.1.1h: Coding loop statements to reduce the number of action lines.

12.1.1i: Utilizing break and continue statements to immediately exit a loop and begin another iteration of any loop respectively.</td></tr>
</table>

	12.1.1j: Using function codes to avoid writing the same codes 12.1.1k:Coding and handling events 12.1.1l: Code cookies into a webpage 12.1.1m: Creating dialog boxes for a webpage 12.1.1n: Implement the print function window object on the webpage		
2. JavaScript Objects	12.1.2a: Create and use the number, boolean, string, array, date, math, and RegExp objects using JavaScript in the webpage 12.1.2b: Selecting and Styling DOM elements in JavaScript 12.1.2c: Get, set and remove attributes from HTML elements using JavaScript 12.1.2d: Manipulating HTML elements using JavaScript 12.1.2e: Navigate between DOM nodes using JavaScript 12.1.2f: Implementing and manipulating the JavaScript Window, Screen, Location, Window History, and Window Navigation Objects 12.1.2g: Using a timer to execute a function at a particular time.		
3. JavaScript (Advance)	12.1.3a: Using JavaScript date and time objects in webpages to get the user's local date and time by accessing the system clock through the browser. 12.1.3b: Perform mathematical operations using JavaScript		

	12.1.3c: Convert the data type of different values using JavaScript 12.1.3d: Using DOM event listeners using JavaScript 12.1.3e: Utilizing the event propagation 12.1.3f: Borrow functionality from existing objects in JavaScript 12.1.3g: Able to gracefully deal with errors 12.1.3h: Validating HTML forms using JavaScript 12.1.3i: Creating animation on your webpage using JavaScript 12.1.3j: Include multimedia on your webpage using JavaScript 12.1.3k: Find and fix bugs with JavaScript 12.1.3l: Creating a client-side mage map		

MODULE 12.2: Software Engineering	Content Standard: 12.2a: Use the Software Development life Cycle and Models on a school software project 12.2b: Organize codes using OOP and/or PP		
Topic	**Performance Indicators**	**Module Benchmark**	**Assessment Tasks**
1. Software Development Life Cycle (SDLC)	**12.2.1a:** Use each phase of the Software Development Life Cycle to tackle a software development project' **12.2.1b:** Use software development idea to plan to solve a problem **12.2.1c:** Appropriately apply the SDLC and a model to a software development scenario		**Test + Case Study**
2. Programming Paradigms - PP vs. OOP	**12.2.2a:** Differentiate between Procedural Programming (PP) and Object-Oriented Programming (OOP) **12.2.2a:** Organizing codes using PP or OOP		

MODULE 12.3: Python	Content Standard: 12.3a: Utilizing Python basic-to-advanced concepts on Software Development applications such a mobile or pc game.		
Topic	**Performance Indicators**	**Module Benchmark**	**Assessment Tasks**
1. Basic Python	**12.3.1a:** Set up the suitable environment to use Python programming **12.3.1b:** Create variables using Python **12.3.1c:** Use different Python operators **12.3.1d:** Use algorithm knowledge with different decision-making statements using Python		**Assignment Test + Quiz**

	12.3.1e: Use algorithm knowledge with loop statements using Python **12.3.1f:** Create strings using Python **12.3.1g:** Construct list data types with Python **12.3.1h:** Create tuples using Python **12.3.1i:** Access dictionary elements using keys **12.3.1j:** Manipulating date and time elements with Python **12.3.1k:** Create a function using Python **12.3.1l:** Produce I/O files using Python **12.3.1m:** Use exception handling and/or assertion to handle any unexpected error in a Python program.		
2. Advance Python	**12.3.2a:** Create a class of objects using Python **12.3.2b:** Creating Instance Objects **12.3.2c:** Accessing object attributes **12.3.2d:** Destroying Objects **12.3.2e:** Creating a CGI program		

	12.3.2f: Creating a database and interfacing your program to the database **12.3.2g:** Utilize socket programming using Python **12.3.2h**: Creating a multithread program **12.3.2i:** Build a software application for your school		

Table 1.2: Mapping content progression by modules

Module	Content Standard	12.1	12.2	12.3
12.1 JavaScript	**12.1**: Utilizing JavaScript basic-to-advanced concepts on Web Development applications	✓		
12.2 Software Development	**12.2a:** Use the Software Development life Cycle and Models on a school software project **12.2b:** Organize codes using OOP and/or PP		✓	
12.3 Python	**12.3:** Utilizing Python basic-to-advanced concepts on Software Development applications such a mobile or pc game.			✓

MODULE 12.1: JAVASCRIPT

3 Weeks

HTML, CSS and JavaScript are three main programming languages for Web Development. After utilizing HTML and CSS to build and style the webpage/website in a static manner, JavaScript is added to provide more dynamics to the web application.

This module aims to give students the basic-to-advance concepts of coding using the JavaScript language. The application surrounding this module is on web development, however, students can be challenged upon teacher's instruction to utilize the programming language for different applications.

The outline of the module is structured in the following manner:
Topic 1: JavaScript (Basic)
Topic 2: JavaScript Objects
Topic 3: JavaScript (Advance)

Linking Modules

Prerequisites:

- STEM-proper
    - Module 11.5 Computer Programming
        - Topic 2: HTML Basics
        - Topic 3: HTML Advance and HTML5 Features
    - Module 11.6 Web Development – Cascading Style Sheets (CSS)

Broad Content Standard

12.1: Utilizing JavaScript basic-to-advanced concepts on Web Development applications

Content Benchmarking

Students' ability to master the knowledge and skills of:

- Creating responsive and interactive elements of a Webpage/Website using programming skills of HTML, CSS and JavaScript.

Topic 1: JavaScript (Basic)

Performance Indicators

Students are expected to:

12.1.1a: Enabling and disabling JavaScript support in browsers
12.1.1b: Placing JavaScript in HTML files in different ways
12.1.1c: Utilizing and/or manipulate the different JavaScript data types to support a software program
12.1.1d: Declare and utilize variables with JavaScript
12.1.1e: Write decision-making code using if…else…elseif conditional statements in JavaScript
12.1.1f: Use switch…case statement to test or evaluate an expression with different values in JavaScript.
12.1.1g: Manipulate or perform the operations on variables and values using the operators in JavaScript
12.1.1h: Coding loop statements to reduce the number of action lines.
12.1.1i: Utilizing break and continue statements to immediately exit a loop and begin another iteration of any loop respectively.
12.1.1j: Using function codes to avoid writing the same codes
12.1.1k: Coding and handling events
12.1.1l: Code cookies into a webpage
12.1.1m: Creating dialog boxes for a webpage
12.1.1n: Implement the print function window object on the webpage

Activity 1: Student Task/Quiz/Test

Prepare a quiz/test/workout sheet to test students.

Q 1 - Which of the following is true about typeof operator in JavaScript?
A - The typeof is a unary operator that is placed before its single operand, which can be of any type.
B - Its value is a string indicating the data type of the operand.
C - Both of the above.
D - None of the above.

Q 2 - Can you pass a anonymous function as an argument to another function?
A - true
B - false

Q 3 - Which built-in method calls a function for each element in the array?
A - while()
B - loop()
C - forEach()
D - None of the above.

Q 4 - Which built-in method returns the calling string value converted to upper case?
A - toUpperCase()
B - toUpper()
C - changeCase(case)
D - None of the above.

Q 5 - Which of the following function of Boolean object returns a string containing the source of the Boolean object?

A - toSource()

B - valueOf()

C - toString()

D - None of the above.

Q 6 - Which of the following function of String object returns the characters in a string beginning at the specified location through the specified number of characters?

A - slice()

B - split()

C - substr()

D - search()

Q 7 - Which of the following function of String object creates a string to be displayed in a big font as if it were in a <big> tag?

A - anchor()

B - big()

C - blink()

D - italics()

Q 8 - Which of the following function of Array object returns a new array comprised of this array joined with other array(s) and/or value(s)?

A - concat()

B - pop()

C - push()

D - some()

Q 9 - Which of the following function of Array object adds one or more elements to the end of an array and returns the new length of the array?

A - pop()

B - push()

C - join()

D - map()

Q 10 - Which of the following function of Array object returns a string representing the array and its elements?

A - toSource()

B - sort()

C - splice()

D - toString()

Resources: NIL

(Refer to TRB)

Topic 2: JavaScript Objects

Performance Indicators
Students are expected to:

- **12.1.2a:** Create and use the number, boolean, string, array, date, math, and RegExp objects using JavaScript in the webpage
- **12.1.2b:** Selecting and Styling DOM elements in JavaScript
- **12.1.2c:** Get, set and remove attributes from HTML elements using JavaScript
- **12.1.2d:** Manipulating HTML elements using JavaScript
- **12.1.2e:** Navigate between DOM nodes using JavaScript
- **12.1.2f:** Implementing and manipulating the JavaScript Window, Screen, Location, Window History, and Window Navigation Objects
- **12.1.2g:** Using a timer to execute a function at a particular time.

Practical Assignment

Exercise 1:
Several variables are declared and initialized in 2 program lines:

```
let city = 'Lubumbashi';
let country = 'Congo';
```

Declare a variable flower and assign it the value 'rose'. Declare a second variable tree and assign it the value 'maple'.

NOTE: More Sample Exercises for this Practical can be found in:
https://www.jshero.net/en/success.html

Topic 3: JavaScript (Advance)

Performance Indicators

Students are expected to:

- **12.1.3a:** Using JavaScript date and time objects in webpages to get the user's local date and time by accessing the system clock through the browser.
- **12.1.3b:** Perform mathematical operations using JavaScript
- **12.1.3c:** Convert the data type of different values using JavaScript
- **12.1.3d:** Using DOM event listeners using JavaScript
- **12.1.3e:** Utilizing the event propagation
- **12.1.3f:** Borrow functionality from existing objects in JavaScript
- **12.1.3g:** Able to gracefully deal with errors
- **12.1.3h:** Validating HTML forms using JavaScript
- **12.1.3i:** Creating animation on your webpage using JavaScript
- **12.1.3j:** Include multimedia on your webpage using JavaScript
- **12.1.3k:** Find and fix bugs with JavaScript
- **12.1.3l:** Creating a client-side mage map

Student Task/Quiz

Q 1 - Which of the following is correct about features of JavaScript?

A - JavaScript is a lightweight, interpreted programming language.

B - JavaScript is designed for creating network-centric applications.

C - JavaScript is complementary to and integrated with Java.

D - All of the above.

Answer: D

Q 2 - Which of the following is the correct syntax to create a cookie using JavaScript?

A - document.cookie = 'key1 = value1; key2 = value2; expires = date';

B - browser.cookie = 'key1 = value1; key2 = value2; expires = date';

C - window.cookie = 'key1 = value1; key2 = value2; expires = date';

D - navigator.cookie = 'key1 = value1; key2 = value2; expires = date';

Answer: A

Q 3 - Which built-in method returns the length of the string?

A - length()

B - size()

C - index()

D - None of the above.

Answer: A

Q 4 - Which built-in method sorts the elements of an array?

A - changeOrder(order)
B - order()
C - sort()
D - None of the above.

Answer: C

Q 5 - Which of the following function of String object returns the character at the specified index?

A - charAt()
B - charCodeAt()
C - concat()
D - indexOf()

Answer D

Q 6 - Which of the following function of String object returns the index within the calling String object of the first occurrence of the specified value?

A - substr()
B - search()
C - lastIndexOf()
D - indexOf()

Answer: D

Q 7 - Which of the following function of String object returns a string representing the specified object?

A - toLocaleUpperCase()
B - toUpperCase()
C - toString()
D - substring()

Answer: C

Q 8 - Which of the following function of String object causes a string to be displayed in the specified size as if it were in a <font size = 'size'> tag?

A - fixed()
B - fontcolor()
C - fontsize()
D - bold()

Answer: C

Q 9 - Which of the following function of Array object joins all elements of an array into a string?

A - concat()

B - join()

C - pop()

D - map()

Answer: B

Q 10 - Which of the following function of Array object represents the source code of an object?

A - toSource()

B - splice()

C - toString()

D - unshift()

Answer: A

Practical Assignment

1. Write a JavaScript program to find the area of a triangle where lengths of the three of its sides are 5, 6, 7.

Sample Solution:

HTML Code

```
<!DOCTYPE html>
<html>
<head>
<meta charset=utf-8 />
<title>The area of a triangle</title>
</head>
<body>
</body>
</html>
```

JavaScript Code

```
var side1 = 5;
var side2 = 6;
var side3 = 7;
var s = (side1 + side2 + side3)/2;
var area = Math.sqrt(s*((s-side1)*(s-side2)*(s-side3))); console.log(area);
```

More Information regarding this exercises and also more exercises similar to this can be found in this link. https://www.w3resource.com/javascript-exercises/javascript-basic-exercises.php (You can use to collect questions for this Practical Assignment.)

SAMPLE ASSESSMENT

Sample Assessment Methods

- Group Assignment
- Presentation
- Projects
- Written or Practical Quiz
- Written or Practical Test
- Practical Assignment
- Case Studies

Sample Assessment Tasks

Module	Content Standard	Topic	Assessment Tasks
12.1:	**12.1.**	**1.**	Written Quiz Written Quiz Project + Written Test

Sample Assessment Marking Guide

Topic	Assessment Task	Assessment Criteria	Total Marks
1	Written Quiz	Explicitly define Agribiotech and other engineering-related terms.	2.5
		Articulate and explain the contrast of the different categories and types of large-base agricultural systems.	2.5
		Adequately display a desire to become subject matter experts by addressing any real-life issues with logical reasoning before coding.	5
		Clearly explain in their own words the fundamental process of AI development.	5
		Clearly differentiate the branches of AI	2.5
		Explicitly define Agricultural Engineering and other engineering-related terms.	2.5
		Subtotal	**20**
Module 12.1 (All four Topics)	Written Test *(Multiple Choices, Short Answers, Calculations, etc)*	Correctly answers all long questions	10
		Correctly answers all multiple choice	30
		Correctly answers short answer questions	40
		Subtotal	***80***
		Module TOTAL	**100**

MODULE 12.2: SOFTWARE DEVELOPMENT

2-3 Weeks

This module introduces students to the process of software development. It encompasses knowledge around the stages of developing a software, types of software development and the possible application of software development to solve problems. Also, included is the two fundamental programming paradigms – Procedural Programming and Object-Oriented Programming.

The SDLC is an acronym for Software Development Life Cycle. It is a sequence of scheduled actions for DEVELOPING or altering Software Products. This Module will provide you an understanding of the SDLC fundamentals, as well as the many SDLC models and their applications. In addition, other related techniques such as Agile, Rapid Application Development, and Prototyping will also be covered.[1]

Programming paradigms refer to a variety of distinct approaches to the programming process. Different paradigms reflect fundamentally different ways to utilizing programming to provide answers to certain sorts of issues. The majority of programming languages belong to one paradigm, although others include components from many paradigms. The procedural and object-oriented paradigms are two of the most important programming paradigms.[2]

The outline of the module is structured in the following manner:
Topic 1: Software Development Life Cycle (SDLF)
Topic 2: PP vs OOP

Linking Modules

Prerequisites: NIL

Broad Content Standard

12.2a: Use the Software Development life Cycle and Models on a school software project
12.2b: Organize codes using OOP and/or PP

Content Benchmarking

Students' ability to master the knowledge and skills of:

- The Software Development Life Cycle and how to use it design a Software
- Organizing codes using Procedural Programming or Object-Oriented Programming

[1] SDLC Tutorial (tutorialspoint.com)
[2] Object-Oriented Programming vs. Procedural Programming - Video & Lesson Transcript | Study.com

Topic 1: Software Development Life Cycle (SDLC)

Performance Indicators
Students are expected to:

- **12.2.1a:** Use each phase of the Software Development Life Cycle to tackle a software development project
- **12.2.1b:** Use software development idea to plan to solve a problem
- **12.2.1c:** Appropriately apply the SDLC and a model to a software development scenario

Quiz/Test/Practical Assignment

1. The approach/document used to make sure all the requirements are covered when writiting test cases.

REQUIREMENT TRACEBILITY		BUSSINESS REQUIREMENTS					
		BID001	BID002	BID003	BID004	BID005	
TEST CASES							

a) Test Matrix
b) Checklist
c) Test Bed
d) Traceability Matrix

Answer: C

2. To check whether we are developing the right product according to the customer requirements.
A) Validation
B) Verification
C) Quality Assurance
D) Quality Control

NOTE: More Questions like this here: https://www.proprofs.com/quiz-school/playquiz/?title=test-2-multiple-choice. You can use any one of them for your Quizes/Tests/Assignment.

Topic 2: Programming Paradigms – PP vs. OOP

Performance Indicators

Students are expected to:

12.2.2a: Differentiate between Procedural Programming (PP) and Object-Oriented Programming (OOP)

12.2.2a: Organizing codes using PP or OOP

Assignment

1. The method with the same name or different return type and difference in the parameters either in number or type is known as:
A. Function overloading
B. Compile Time Overloading

Answer: B

2. To call a base class constructor in a derived class, is it needed to call the base class initializer.
A. True
B. False

Answer: B

3. Can objects of abstract classes be instantiated?
A. True
B. False

Answer: B

4. The process by which one object can acquire the properties of another object:
A. Encapsulation
B. Inheritance
C. Polymorphism

Answer: B

5. Constructors are used to:
A. To build a user interface.
B. Free memory.
C. Initialize a newly created object.
D. To create a sub-class.

Answer: C

6. An object that has more than one form is referred to as:
A. Inheritance
B. Interface
C. Abstract class
D. Polymorphism

Answer: D

7. Information Hiding can also be termed as:
A. Data hiding
B. Encapsulation
C. Inheritance

Answer: B

8. Pick the term that relates to polymorphism:
A. Dynamic binding
B. Dynamic allocation
C. Static typing
D. Static allocation

Answer: A

9. Main method can be overridden:
A. True
B. False

Answer: B

10. The keyword which is used to access the method or member variables from the superclass:
A. Super
B. Using
C. Is_a
D. Has_a

Answer: A

11. What modifiers are allowed for methods in an Interface? (choose one or more answers)
A. Public
B. Private
C. Abstract

Answer: A and C

12. When sub class declares a method that has the same type of arguments as a method declared by one of its superclasses, it is termed as:
A. Method overriding
B. Method overloading
C. Operator overloading
D. Operator overriding

Answer: A

NOTE: More Questions like this in https://www.proprofs.com/quiz-school/playquiz/?title=object-oriented-programming-basics

SAMPLE ASSESSMENT

Sample Assessment Methods

- Group Assignment
- Presentation
- Projects
- Written or Practical Quiz
- Written or Practical Test
- Practical Assignment
- Case Studies

Sample Assessment Tasks

Module	Content Standard	Topic	Assessment Tasks
12.1:	**12.1.**	**2.**	Written Quiz Written Quiz Project + Written Test

Sample Assessment Marking Guide

Topic	Assessment Task	Assessment Criteria	Total Marks
1	Written Quiz	Explicitly define Agribiotech and other engineering-related terms.	2.5
		Articulate and explain the contrast of the different categories and types of large-base agricultural systems.	2.5
		Adequately display a desire to become subject matter experts by addressing any real-life issues with logical reasoning before coding.	5
		Clearly explain in their own words the fundamental process of AI development.	5
		Clearly differentiate the branches of AI	2.5
		Explicitly define Agricultural Engineering and other engineering-related terms.	2.5
		Subtotal	**20**
Module 12.1 (All four Topics)	Written Test *(Multiple Choices, Short Answers, Calculations, etc)*	Correctly answers all long questions	10
		Correctly answers all multiple choice	30
		Correctly answers short answer questions	40
		Subtotal	***80***
		Module TOTAL	**100**

MODULE 12.3: PYTHON

2-3 Weeks

Python is a general-purpose programming language, which means it can be used to create almost anything with the appropriate tools and libraries. It is ideal for backend web development, data analysis, artificial intelligence, and scientific computing in the professional world. It has also been used to create productivity tools, games, and desktop programs, so there are lots of resources to assist you learn how to do so.

This module aims to give students the basic-to-advance concepts of coding using the Python programming language. The application surrounding this module is on software development, particularly a simple app or video game. However, students can be challenged upon teacher's instruction to utilize the programming language for different applications.

The outline of the module is structured in the following manner:
Topic 1: Basic Python
Topic 2: Advance Python

Linking Modules
Link:

- STEM-proper
    - Module 11.4 Algorithms and Data Structures
    - Module 12.2 Software Development

Prerequisites: Basic Computer programming knowledge

Broad Content Standard
12.3a: Utilizing Python basic-to-advanced concepts on Software Development applications such a mobile or pc game.

Content Benchmarking
Students' ability to master the knowledge and skills of:

- Basic and Advance Python concepts
- Building a software application

Topic 1: Basic Python

Performance Indicators

Students are expected to:

- **12.4.1a:** Set up the suitable environment to use Python programming
- **12.4.1b:** Create variables using Python
- **12.4.1c:** Use different Python operators
- **12.4.1d:** Use algorithm knowledge with different decision-making statements using Python
- **12.4.1e:** Use algorithm knowledge with loop statements using Python
- **12.4.1f:** Create strings using Python
- **12.4.1g:** Construct list data types with Python
- **12.4.1h:** Create tuples using Python
- **12.4.1i:** Access dictionary elements using keys
- **12.4.1j:** Manipulating date and time elements with Python
- **12.4.1k:** Create a function using Python
- **12.4.1l:** Produce I/O files using Python
- **12.4.1m:** Use exception handling and/or assertion to handle any unexpected error in a Python program.

Topic 2: Advance Python

Performance Indicators

12.4.2a: Create a class of objects using Python
12.4.2b: Creating Instance Objects
12.4.2c: Accessing object attributes
12.4.2d: Destroying Objects
12.4.2e: Creating a CGI program
12.4.2f: Creating a database and interfacing your program to the database
12.4.2g: Utilize socket programming using Python
12.4.2h: Creating a multithread program
12.4.2i: Build a software application for your school

Lab 1: Using the Interpreter

1. *Interaction*. Using a system command-line, IDLE, or other, start the Python interactive command line (>>> prompt), and type the expression: "Hello World!" (including the quotes). The string should be echoed back to you. The purpose of this exercise is to get your environment configured to run Python. In some scenarios, you may need to first run a cd shell command, type the full path to the python executable, or add its path to your PATH environment variable. Set it in your .cshrc or .kshrc file to make Python permanently available on Unix systems; use a setup.bat, autoexec.bat, or the environment variable GUI on Windows.

2. *Programs*. With the text editor of your choice, write a simple module file—a file containing the single statement: print 'Hello module world!'. Store this statement in a file named module1.py. Now, run this file by using any launch option you like: running it in IDLE, clicking on its file icon, passing it to the Python interpreter program on the system shell's command line, and so on. In fact, experiment by running your file with as many of the launch techniques we've seen in this unit as you can. Which technique seems easiest (there is no right answer to this one, of course)?

3. *Modules*. Next, start the Python interactive command line (>>> prompt) and import the module you wrote in the prior exercise. Does your PYTHONPATH setting need to include the directory where the file is stored? Try moving the file to a different directory and importing it again from its original directory; what happens? (Hint: is there still a file named module1.pyc in the original directory?)

4. *Scripts*. If your platform supports it, add the #! line to the top of your module1.py module, give the file executable privileges, and run it directly as an executable. What does the first line need to contain? Skip that if you are working on a Windows machine (#! usually only has meaning on Unix and Linux); instead try running your file by listing just its name in a DOS console window (this works on recent flavors of Windows), or the "Start/Run…" dialog box.

5. *Errors*. Experiment with typing mathematical expressions and assignments at the Python interactive command line. First type the expression: 1 / 0; what happens? Next, type a variable name you haven't assigned a value to yet; what happens this time?

You may not know it yet, but you're doing exception processing, a topic we'll explore in depth in a later unit. As we'll learn then, you are technically triggering what's known as the *default exception handler*—logic that prints a standard error message.

For full-blown source-code *debugging* chores, IDLE includes a GUI debugging interface (select Debug before running your script), and a Python standard library module named pdb provides a command-line debugging interface (more on pdb later). When first starting out, though, Python's default error messages will probably be as much error handling as you need—they give the cause of the error, as well as the lines in your code were active when the error occurred.

6. *Breaks*. At the Python command line, type:

```
L = [1, 2]
L.append(L)
L
```

What happens? If you're using a Python newer than release 1.5, you'll probably see a strange output. If you're using a Python version older than 1.5.1 (now ancient history!), a Ctrl-C key combination will probably help on most platforms. Why do you think this occurs? What does Python report when you type the Ctrl-C key combination? Warning: if you do have a Python older than release 1.5.1, make sure your machine can stop a program with a break-key combination of some sort before running this test, or you may be waiting a long time.

7. *Documentation*. Spend at least 6 minutes browsing the Python library and language manuals before moving on, to get a feel for the available tools in the standard library, and the structure of the documentation set. It takes at least this long to become familiar with the location of major topics in the manual set; once you do, though, it's easy to find what you need. You can find this manual in the Python "Start" button entry on Windows, in the "Help" pulldown menu in IDLE, or online at www.python.org. We'll also have a few more words to say about the manuals, and other documentation sources available (including PyDoc and the help function), in the statements unit. If you still have time to kill, go explore the Python website (www.python.org), and the Vaults of Parnassus site. Especially check out the python.org documentation and search pages; they can be crucial resources in practice.

Solution

1. *Interaction*. Assuming your Python is configured properly, you should participate in an interaction that looks something like the following. You can run this any way you like: in IDLE, from a shell prompt, and so on.

Note: lines that start with a "**%**" denote shell-prompt command lines; don't run these lines at Python's "**>>>**" prompt, and don't type either the "**%**" or "**>>>**" characters yourself—enter just the code after the these prompts. Run "**%**" shell-prompt commands in a Command Prompt window on Windows, and run "**>>>**" commands in Python (IDLE's shell window, etc.). You don't need to run the first command that follows if you're working only in IDLE, and you may need to use a full "C:\Python35\python" instead of just "python" if Python isn't on your system's PATH setting:

```
% python                # "%" means your shell prompt (e.g., "C:\code>")
...copyright information lines...
>>> "Hello World!"      # ">>>" is Python's prompt: Python code goes here
'Hello World!'
>>>                     # Ctrl-D, Ctrl-Z, or window close to exit
```

2. *Programs*. Here's what your code (i.e., module) file and shell interactions should look like; again, feel free to run this other ways—by clicking its icon, by IDLE's Edit/RunScript menu option, and so on:

Note: in this section a "**File: xxx.py**" in italics gives the name of the file in which code following it is to be stored, and be sure to always use the parenthesized call form "**print(xxx)**" if you're using Python 3.X (see the statements unit for more details).

```
# File: module1.py              # enter this code in a new file
print 'Hello module world!'     # 3.X: use the form print('…')
```

```
% python module1.py            # run this command line at a system prompt
Hello module world!
```

3. *Modules*. The following interaction listing illustrates running a module file by importing it. Remember that you need to reload it to run again without stopping and restarting the interpreter. The bit about moving the file to a different directory and importing it again is a trick question: if Python generates a module1.pyc file in the original directory, it uses that when you import the module, even if the source code file (.py) has been moved to a directory not on Python's search path. The .pyc file is written automatically if Python has access to the source file's directory and contains the compiled bytecode version of a module. We look at how this works again in the modules unit.

```
% python
>>> import module1
Hello module world!
>>>
```

4. *Scripts*. Assuming your platform supports the #! trick, your solution will look like the following (though your #! line may need to list another path on your machine):

```
File: module1.py
#!/usr/local/bin/python          (or #!/usr/bin/env python)
print 'Hello module world!'

% chmod +x module1.py

% module1.py
Hello module world!
```

5. *Errors*. The interaction below demonstrates the sort of error messages you get if you complete this exercise. Really, you're triggering Python exceptions; the default exception handling behavior terminates the running Python program and prints an error message and stack trace on the screen. The stack trace shows where you were at in a program when the exception occurred (it's not very interesting here, since the exceptions occur at the top level of the interactive prompt; no function calls were in progress). In the exceptions unit, you will see that you can catch exceptions using "try" statements and process them arbitrarily; you'll also see that Python includes a full-blown source-code debugger for special error detection requirements. For now, notice that Python gives meaningful messages when programming errors occur (instead of crashing silently):

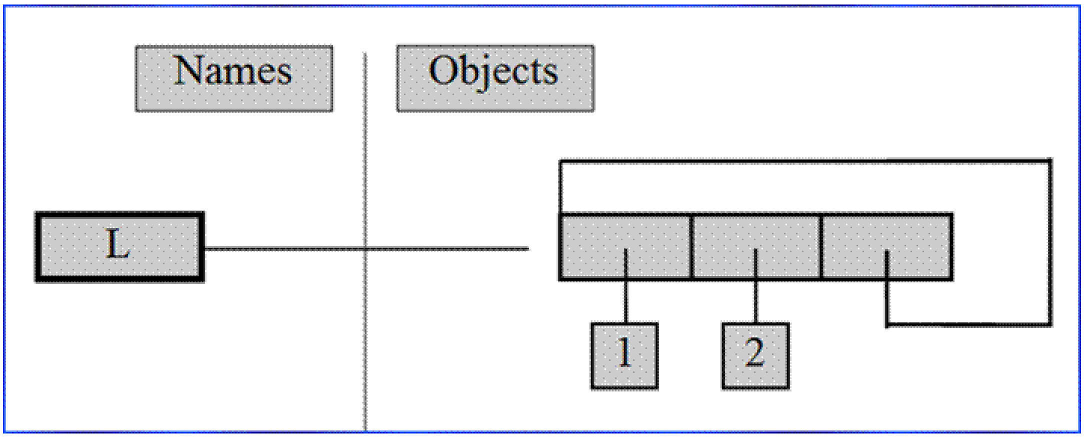
```
% python
>>> 1 / 0
Traceback (innermost last):
  File "<stdin>", line 1, in ?
ZeroDivisionError: integer division or modulo
>>>
>>> x
Traceback (innermost last):
  File "<stdin>", line 1, in ?
NameError: x
```

6. *Breaks*. When you type this code:

```
L = [1, 2]
L.append(L)
```

you create a cyclic data-structure in Python. In Python releases before Version 1.5.1, the Python printer wasn't smart enough to detect cycles in objects, and it would print an unending stream of [1, 2, [1, 2, [1, 2, [1, 2, and so on, until you hit the break key combination on your machine (which, technically, raises a keyboard-interrupt exception that prints a default message at the top level unless you intercept it in a program). Beginning with Python Version 1.5.1, the printer is clever enough to detect cycles and prints [[...]] instead to let you know.

The reason for the cycle is subtle and requires information you'll gain in the next unit. But in short, assignment in Python always generates references to objects (which you can think of as implicitly followed pointers). When you run the first assignment above, the name L becomes a named reference to a two-item list object. Now, Python lists are really arrays of object references, with an append method that changes the array in-place by tacking on another object reference. Here, the append call adds a reference to the front of L at the end of L, which leads to the cycle illustrated in the figure below. Believe it or not, cyclic data structures can sometimes be useful (but maybe not when printed!). Today, Python can also reclaim (garbage collect) such objects cyclic automatically.



A cyclic list

SAMPLE ASSESSMENT

Sample Assessment Methods

- Group Assignment
- Presentation
- Projects
- Written or Practical Quiz
- Written or Practical Test
- Practical Assignment
- Case Studies

Sample Assessment Tasks

Module	Content Standard	Topic	Assessment Tasks
12.1:	**12.1.**	**3.**	Written Quiz Written Quiz Project + Written Test

Sample Assessment Marking Guide

Topic	Assessment Task	Assessment Criteria	Total Marks
1	Written Quiz	Explicitly define Agribiotech and other engineering-related terms.	2.5
		Articulate and explain the contrast of the different categories and types of large-base agricultural systems.	2.5
		Adequately display a desire to become subject matter experts by addressing any real-life issues with logical reasoning before coding.	5
		Clearly explain in their own words the fundamental process of AI development.	5
		Clearly differentiate the branches of AI	2.5
		Explicitly define Agricultural Engineering and other engineering-related terms.	2.5
		Subtotal	**20**
Module 12.1 (All four Topics)	Written Test *(Multiple Choices, Short Answers, Calculations, etc)*	Correctly answers all long questions	10
		Correctly answers all multiple choice	30
		Correctly answers short answer questions	40
		Subtotal	***80***
		Module TOTAL	**100**

PLANNING, REPORTING & RECORDING (PERFORMANCE CRITERIA)

Assessment Planning

Assessment Domains	Modules 6	Total Marks	% Weight of Modules
Theory Knowledge & skills – **Test**	Test (50)	/50	**20**
Application of knowledge – **Assignments and projects**	Assign.1 (30)	/30	**30**
End of Term **Exam**	(25 marks)	/25	**50**
		Total marks for term one	**100 marks**

Assessment Recording

Percentage Range	Achievement Level
85 -100	Very High Achievement (VHA)
65-84	High Achievement (HA)
50-64	Satisfactory Achievement (SA)
25-49	Low Achievement (LA)
0-24	Below Minimum Achievement (BMA)

		Assessment Task													
#	**Names**	**Tests**		**Assignments**				Project	**Exam (/50)**	**Total Score (/300)**	**Percentage Weight (%)**				**Achievement Level [VHA, HA, SA,LA]**
		1	**2**	**1**	**2**	**3**	**4**	1			**T (20%)**	**A + P (30%)**	**Exam (50%)**	**Ttl CA**	
1	Bill Joe	45	47	20	21	22	23	55	45	***278***	18.4	25.88	45	***89.28***	VHA
2	Cara Den	20	25	15	16	17	18	25	30	***166***	9	18.22	30	***57.22***	SA
3	Lea May	9	15	5	6	7	8	30	25	***105***	4.8	9.10	25	***38.9***	LA

Assessment Reporting

Thema / Biotechnology / Module 12.1: Protein Sequencing
Broad Content Standard for Assignments / project
12.1 To determine the amino acid sequence of a protein

Performance Criteria	Determine Protein Sequence (15 marks)	Prepare Lab Report (5 marks)	Answers all Relative Questions to protein sequencing (5 marks)	25 marks
Very High Achievement	Accurately determine protein sequence (13-15)	Accurately writes a lab report with correct sections (5)	Accurately answers all relative questions to protein sequencing (5)	**(21-25)**
High Achievement	Partially determine protein sequence (10-12)	Partially writes a lab report with correct sections (4)	Partially answers all relative questions to protein sequencing (4)	**(16-20)**
Satisfactory Achievement	Fairly determine protein sequence (7-9)	Fairly writes a lab report with correct sections (3)	Impartially answers all relative questions to protein sequencing (3)	**(11-15)**
Low Achievement	Poorly determine protein sequence (4-6)	Poorly writes a lab report with correct sections (2)	Poorly answers all relative questions to protein sequencing (2)	**(6-10)**
Below minimum Achievement	Needs improvement in determining protein sequence (0-3)	Needs improvement in writing a lab report with correct sections (1)	Needs improvement in answering all relative questions to protein sequencing (1)	**(0–5)**

Thema Biotechnology / Module 12.1: Protein Sequencing Broad Content Standards For Test: **12.1** To determine the amino acid sequence of a protein				
Performance Criteria	**Correctly answers all long questions (25 marks)**	**Correctly answers all multiple choice (10 marks)**	**Correctly use concepts to design and propose solutions(15 marks)**	**50 marks**
Very High Achievement	Accurately answers all long questions (21-25)	Accurately answers all multiple choice (9-10)	Accurately use concepts to design and propose solutions (13-15)	**(41-50)**
High Achievement	Partially answers all long questions (16-20)	Partially answers all multiple choice (7-8)	Partially use concepts to design and propose solutions (10-12)	**(31-40)**
Satisfactory Achievement	Fairly answers all long questions (11-15)	Fairly answers all multiple choice (5-6)	Fairly use concepts to design and propose solutions (7-9)	**(21-30)**
Low Achievement	Poorly answers all long questions (6-10)	Poorly answers all multiple choice (3-4)	Poorly use concepts to design and propose solutions (4-6)	**(11-20)**
Below minimum Achievement	Needs improvement in answering all long questions (0-5)	Needs answers all multiple choice (0-2)	Needs improvement in using concepts to design and propose solutions (0-3)	**(1-10)**

REFERENCES

1. https://www.javatpoint.com/javascript-tutorial
2. https://www.tutorialrepublic.com/javascript-tutorial/javascript-form-validation.php
3. https://www.tutorialspoint.com/javascript/index.htm
4. https://www.w3schools.com/js/DEFAULT.asp
5. https://www.javascripttutorial.net
6. https://javascript.info
7. https://www.javatpoint.com/javascript-tutorial
8. https://www.geeksforgeeks.org/javascript-tutorial
9. https://www.guru99.com/interactive-javascript-tutorials
10. https://www.w3schools.com/php
11. https://www.tutorialspoint.com/php/index.htm
12. https://www.javatpoint.com/php-tutorial
13. https://www.phptpoint.com/php-tutorial
14. https://www.tutorialspoint.com/software
15. https://www.tutorialspoint.com/sdlc/index.htm
16. https://www.javatpoint.com/software-engineering...
17. https://www.w3schools.in/sdlc-tutorial/software-development-life-cycle-sdlc
18. https://www.guru99.com/php-vs-javascript.html
19. https://www.tutorialspoint.com/php/index.htm
20. https://patriciasdesignsite.wordpress.com/2015/02/19/the-difference-between-html-css-javascript-and-php-languages/
21. https://www.php.net/manual/en/intro-whatis.php#:~:text=PHP%20%28recursive%20acronym%20for%20PHP%3A%20Hypertext%20Preprocessor%29%20is,into%20HTML.%20Nice%2C%20but%20what%20does%20that%20mean%3F
22. https://www.tutorialspoint.com/sdlc/sdlc_overview.htm
23. https://www.tutorialspoint.com/sdlc/index.htm
24. https://study.com/academy/lesson/object-oriented-programming-vs-procedural-programming.html
25. https://stackoverflow.com/questions/1530868/simple-explanation-php-oop-vs-procedural
26.